

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

Лекция 3

15 сентября 2025

Векторизация текста

Зачем нужна векторизация?

- Компьютер «понимает» только числа, а не слова
- Текст нужно преобразовать в числовые признаки
- Векторы позволяют сравнивать слова и анализировать их связи

Простые методы векторизации

One-hot encoding: каждое слово — уникальный индекс, вектор из нулей и одной единицы

- + Простота
- Огромные и разреженные векторы, не учитывают смысловую близость

Bag of Words (BoW): вектор частот слов

- + Учитывает наличие слов
- Теряется порядок, контекст и значение

Ограничения простых методов векторизации

- Невозможно уловить синонимы и многозначность
- Не понимают контекст
- «кот» и «собака» одинаково далеки, как «кот» и «стол»

Word embedding

Эмбеддинг - это способ преобразования чего-то абстрактного в набор чисел и векторов

Ключевая идея эмбеддингов

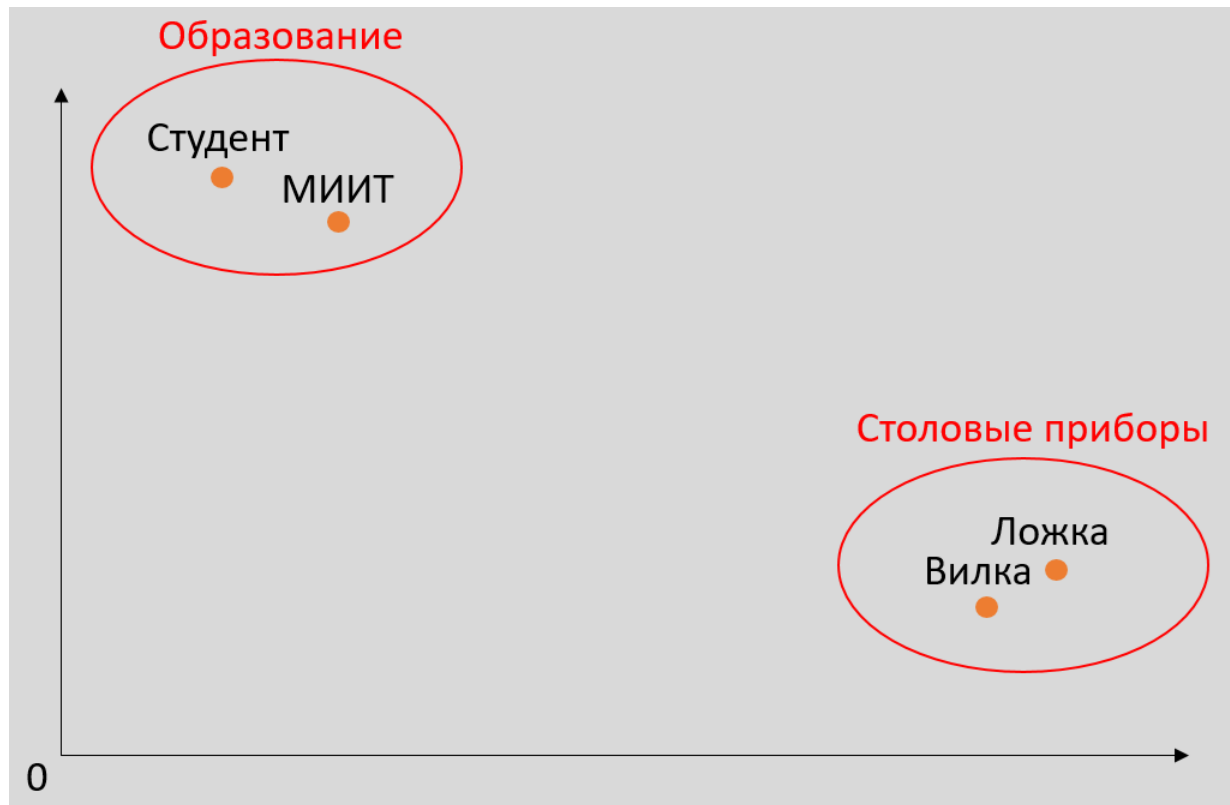
- Представлять слова как плотные векторы в многомерном пространстве
- Слова с близким смыслом имеют близкие вектора

Визуализация в пространстве



Визуализируем
векторы
размерностью 2

Кластеризация



Полученные в
результате
векторизации
эмбеддинги можно
кластеризовать
классическими
методами

Значение контекста

Многозначные слова

Одно и то же слово может иметь разные значения:

- «коса» = причёска
- «коса» = инструмент для покоса травы
- «коса» = песчаная отмель

Только контекст позволяет определить смысл

Разные вектора в зависимости от окружения

Простые модели (BoW, ONE) не различают значения

Современные методы обучают разные представления для одного слова в **разных контекстах**

Пример:

«Девушка заплела косу»: вектор ближе к «челка», «причёска»

«Отец наточил косу»: вектор ближе к «грабли», «серп»

Подходы к учёту контекста

Статистические модели:

- Учитывают фиксированное окно соседних слов
- Ограничены по длине контекста

Нейросетевые модели:

- Word2Vec частично использует локальный контекст
- Современные LM (ELMo, BERT, GPT) строят вектор в зависимости от всего предложения

Учет контекста



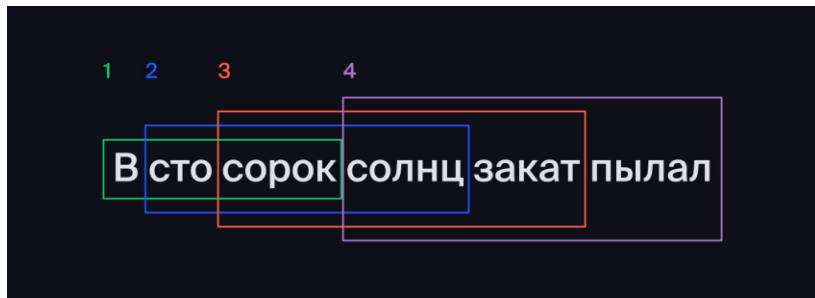
В зависимости от контекста вектор слова “Коса” может иметь разное значение

N-граммы

N-грамма – последовательность из N подряд идущих слов или символов

«машинное обучение полезно»:

- Униграммы: «машинное», «обучение»
- Биграммы: «машинное обучение», «обучение полезно»
- Триграммы: «машинное обучение полезно»



Пример

Я учусь в Российском университете транспорта

Биграммы	Триграммы

Пример

Я учусь в Российском университете транспорта

Биграммы	Триграммы
• Я учусь • учусь в • в Российском • Российском университете • университете транспорта	• Я учусь в • учусь в Российском • В Российском университете • Российском университете транспорта

Языковая модель

Языковая модель — это алгоритм, который умеет оценивать, насколько естественным является текст, и предсказывать следующее слово

Применение:

- Автодополнение текста (телефон, поисковик)
- Проверка правильности текста
- Основа для машинного перевода и чат-ботов

Цепи Маркова

Цепь Маркова — это случайный процесс, в котором будущее состояние зависит только от текущего, а не от всей прошлой истории

Марковское свойство:

$$P(X_{t+1} \mid X_t, X_{t-1}, \dots, X_0) = P(X_{t+1} \mid X_t)$$

Цепи Маркова и N-граммы

see what I found
you found a penny
it has been found
the book you found
you came yesterday

$$P(\text{you, found}) = P(\text{found} \mid \text{you})$$

$$= \text{count}(\text{you found}) / \text{count}(\text{you}) = 2/3$$

Цепи Маркова и N-граммы

1 2 3 4 5 6 7 8

The quick brown fox jumped over the ?

Unigram: $P(w_8)?$

The quick brown fox jumped over **the** ?

Bigram: $P(w_8|w_7)?$

The quick brown fox jumped **over the** ?

Trigram: $P(w_8|w_7, w_6)?$

Проблемы N-грамм

Какие могут проблемы и трудности при использовании N-грамм?

Проблемы N-грамм

Основные ограничения

- Комбинаторный взрыв: количество возможных N-грамм растёт экспоненциально
- Разреженность: многие комбинации слов встречаются редко или вообще не встречаются
- Ограниченный контекст: биграммы и триграммы «видят» только локальное окно, но не понимают дальних зависимостей

Перплексия (perplexity)

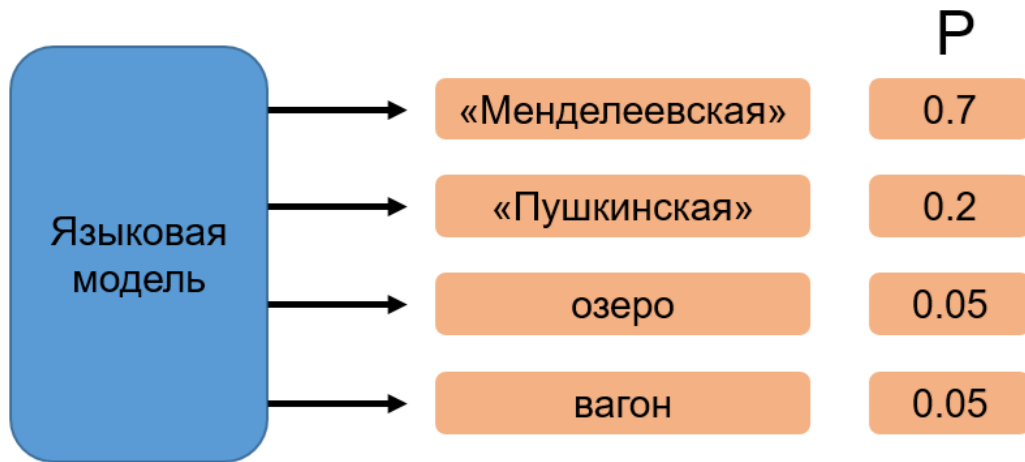
Перплексия — это метрика, используемая для оценки языковых моделей

- Перплексия измеряет, насколько «сильно затрудняется» языковая модель при встрече с тестовым текстом.
- Чем ниже перплексия, тем лучше модель предсказывает слова

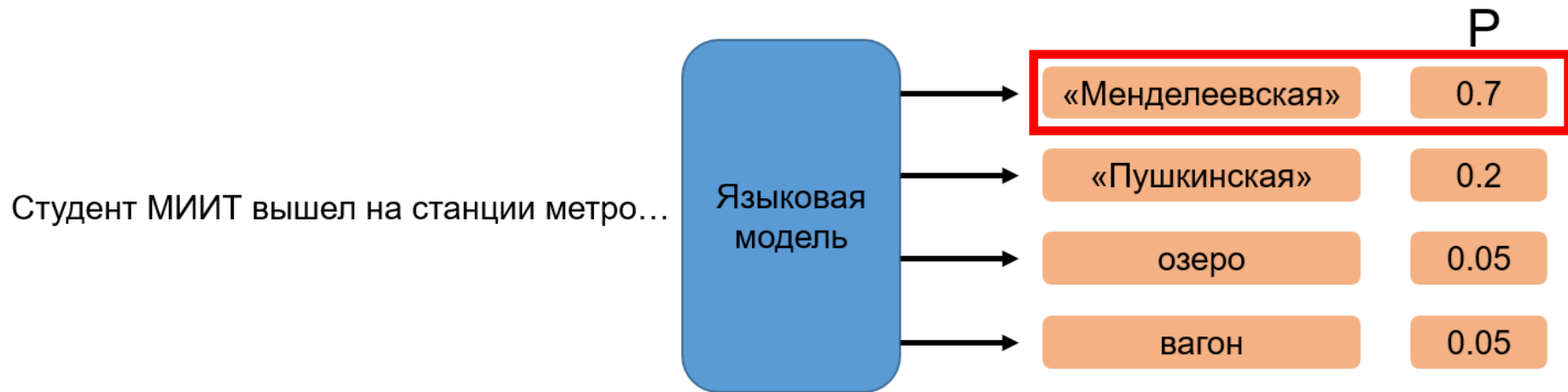
$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_1, \dots, w_{i-1}) \right)$$

Перплексия (perplexity)

Студент МИИТ вышел на станции метро...



Перплексия (perplexity)



Перплексия (perplexity)

Студент МИИТ вышел на станции метро «Менделеевская»
0.5 0.3 0.7 0.9 0.5 0.8 0.7

$$PP(W) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_1, \dots, w_{i-1}) \right)$$

Перплексия (perplexity)

Студент МИИТ вышел на станции метро «Менделеевская»

0.5

0.3

0.7

0.9

0.5

0.8

0.7

$$e^{-\frac{\log(0.5)+\log(0.3)+\log(0.7)+\log(0.9)+\log(0.5)+\log(0.8)+\log(0.7)}{7}}$$

$$e^{0.52} = 1.68$$

Перплексия (perplexity)

Студент МИИТ вышел на станции метро «Менделеевская»

0.5

0.3

0.7

0.9

0.5

0.8

0.7

$$e^{-\frac{\log(0.5)+\log(0.3)+\log(0.7)+\log(0.9)+\log(0.5)+\log(0.8)+\log(0.7)}{7}}$$

$$e^{0.52} = 1.68$$

В среднем модель выбирала между 1.68 равнозначными вариантами

Перплексия (perplexity)

Студент МИИТ вышел на станции метро «Менделеевская»

1 1 1 1 1 1 1

Чему будет равна перплексия?

Перплексия (perplexity)

Студент МИИТ вышел на станции метро «Менделеевская»

1 1 1 1 1 1 1

$$e^{-\frac{\log(1)+\log(1)+\log(1)+\log(1)+\log(1)+\log(1)+\log(1)}{7}}$$

$$e^0 = 1$$

Перплексия (perplexity)

В нейросетевых моделях (LSTM, Transformer) перплексия используется как стандартная метрика качества

- PP = 1: идеал
- PP = 3: нормальная модель
- PP = 100: плохая модель, почти случайное угадывание

Word2Vec

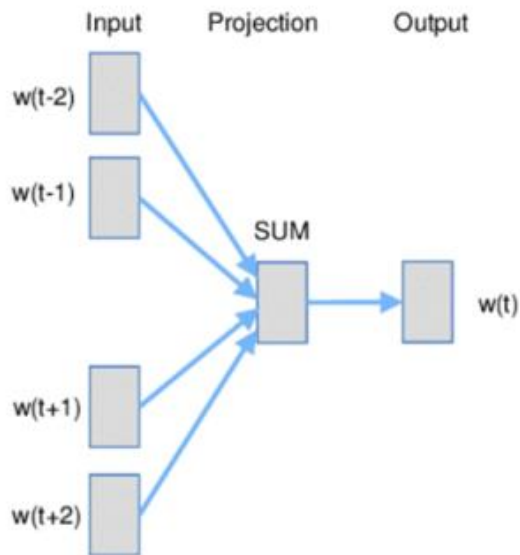
Word2Vec — это метод обучения векторных представлений слов (эмбеддингов), предложенный Google в 2013 году

Основная концепция

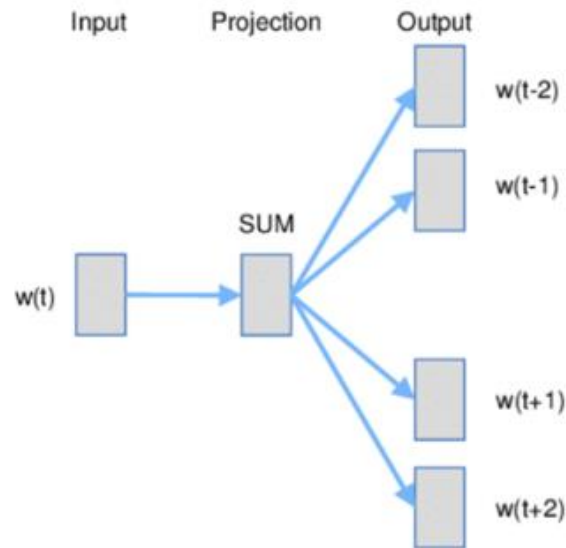
- Слова, встречающиеся в схожих контекстах, имеют близкий смысл
- Задача: обучить вектора так, чтобы они отражали статистику контекстов
- Word2Vec преобразует дискретные слова в плотные векторы (эмбеддинги)

Виды архитектур Word2Vec

CBOW



Skip-gram



Word2Vec

Где взять данные для обучения?

Будем считать, что прогнозируемое слово — «вышел», а соседи — ближайшие от него слова с двух сторон: «студент», «МИИТ» и «на», «станции». Все вместе они образуют пятиграмму:

Студент МИИТ **вышел** на станции метро «Менделеевская»

Word2Vec

Получаем четыре пары слов-соседей:

- вышел — студент
- вышел — МИИТ
- вышел — на
- вышел — станции

Word2Vec

Для следующего слова будет свое контекстное окно

Студент МИИТ вышел **на** станции метро «Менделеевская»

- на — МИИТ
- на — вышел
- на — станции
- на — метро

Word2Vec

Следующая задача word2vec — научить модель отличать истинные пары соседей от случайных

Положительные примеры мы уже собрали. Чтобы получить отрицательные, выберем случайные слова из всего корпуса текста и сделаем из них пары:

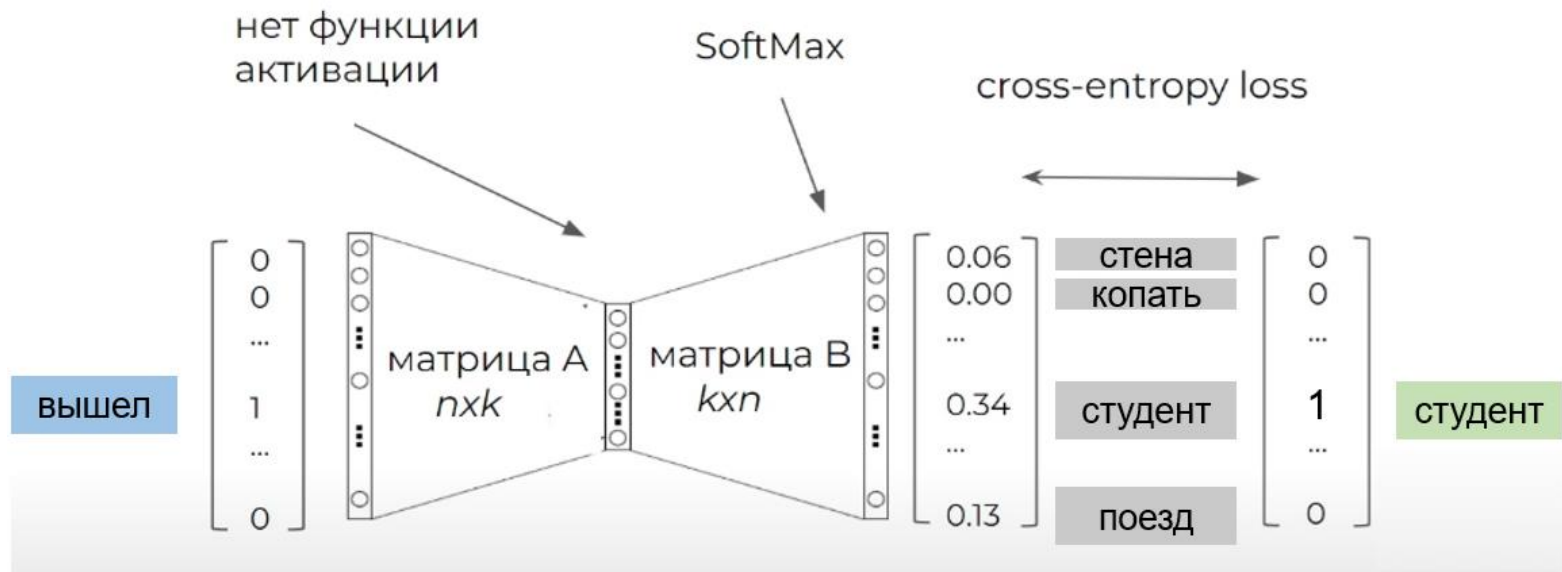
- вышел — «Менделеевская»
- вышел — поезд
- на — студент
- на — университет

Word2Vec

Первое слово	Второе слово	Соседи
вышел	студент	1
вышел	станции	1
вышел	поезд	0
на	станции	1
на	метро	1
на	студент	0

Word2Vec

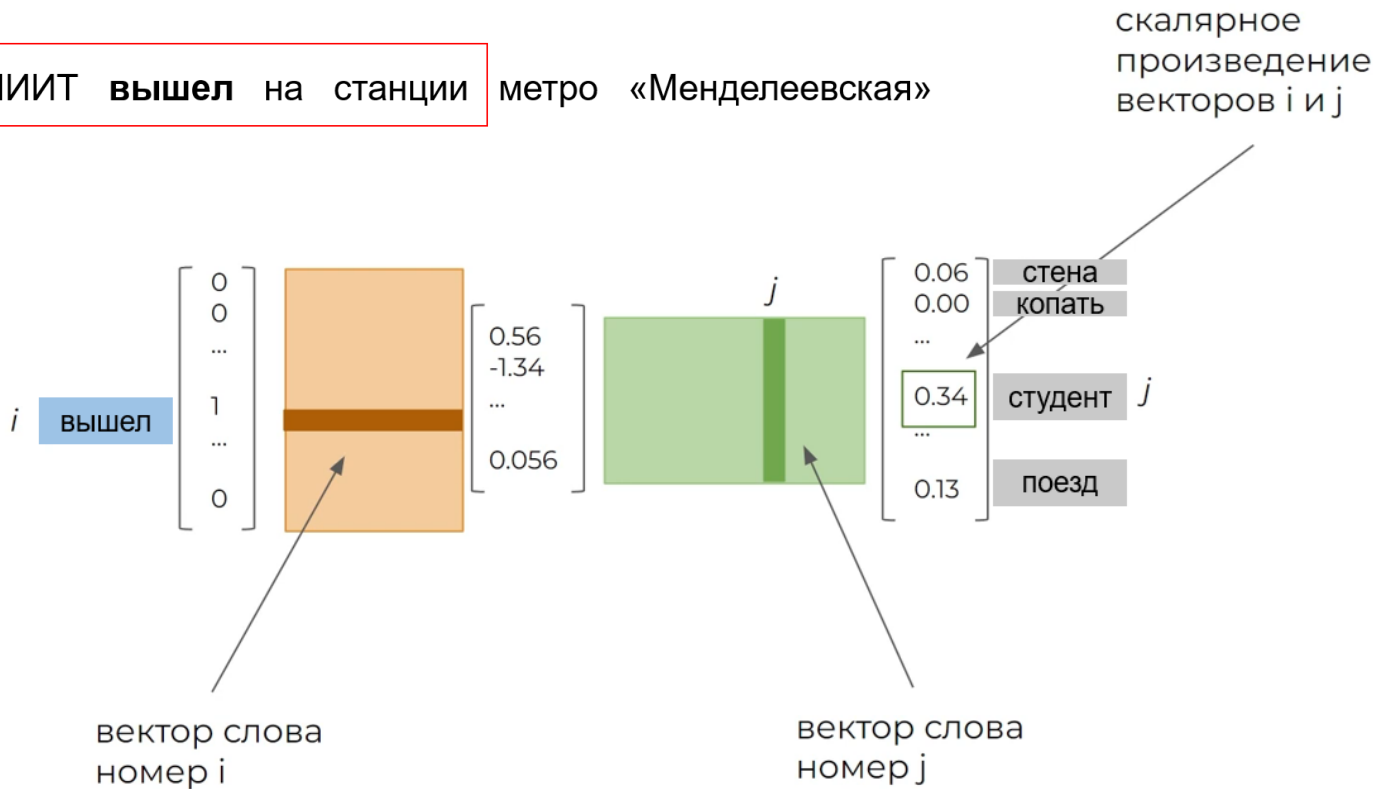
Студент МИИТ **вышел** на станции метро «Менделеевская»



n — размер словаря, k — длина вектора эмбединга (задаем сами)

Word2Vec

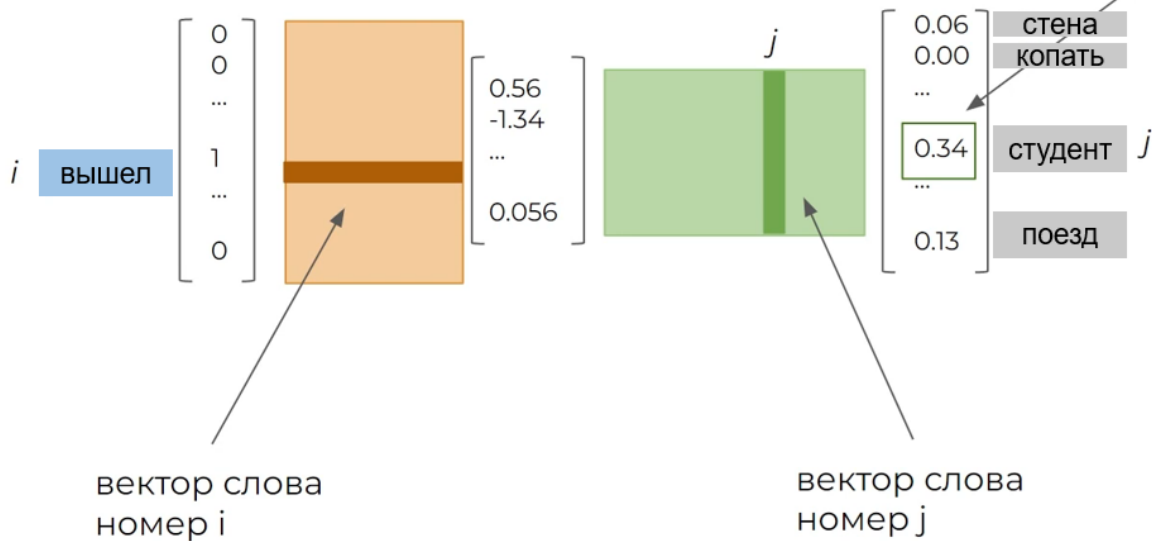
Студент МИИТ **вышел** на станции метро «Менделеевская»



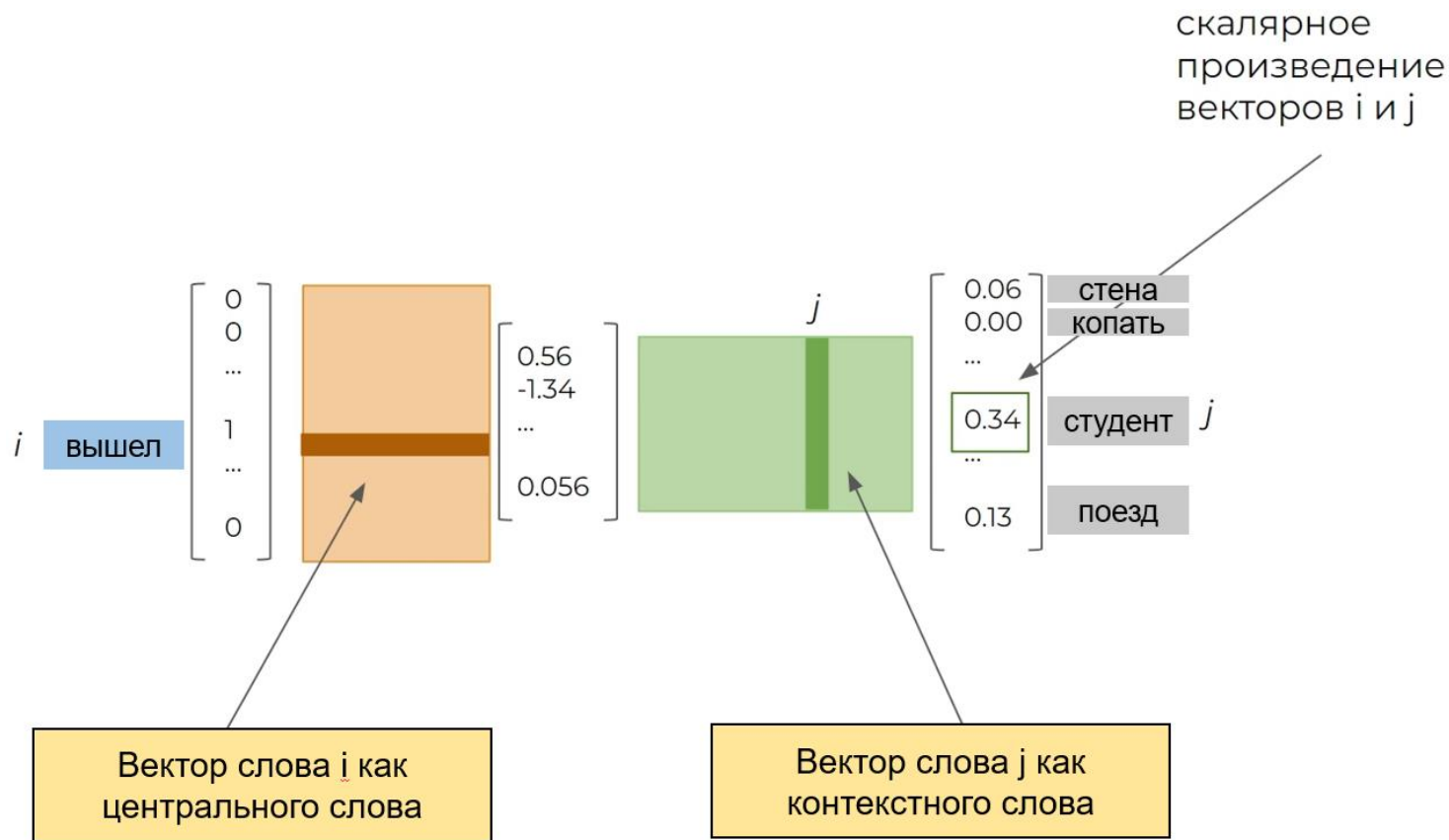
Word2Vec

Нейросеть учит такие векторы слов, чтобы скалярное произведение между векторами двух слов, которые часто используются в одном контексте, были как можно больше

скалярное
произведение
векторов i и j



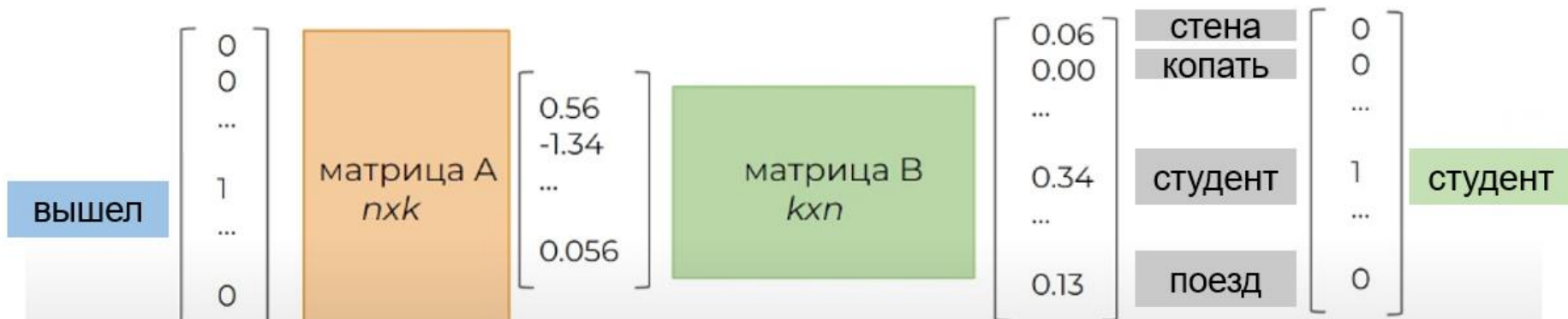
Word2Vec



SkipGram и CBOW

SkipGram — предсказание слов контекста по центральному слову

Студент МИИТ **вышел** на станции метро «Менделеевская»



SkipGram и CBOW

CBOW (Continuous Bag Of Words) — предсказание центрального слова по словам контекста

Студент МИИТ **вышел** на станции метро «Менделеевская»



Функция потерь Word2Vec

Функция потерь при обучении — **кросс-энтропия (Cross-Entropy Loss)**

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

- y_i - правильный ответ
- $\hat{y}_i$ - прогноз модели

Кросс-энтропия

Допустим, наша модель предсказывает один к какому из трех классов относится объект. Если на вход подается объект второго класса, то вектор его правильного предсказания:

$$y = [0, 1, 0]$$

Предсказание модели может выглядеть примерно так:

$$\hat{y} = [0.1, 0.7, 0.2]$$

Чему равна функция потерь кросс-энтропия?

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Кросс-энтропия

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

$$\begin{aligned} \text{Loss} &= - (0 \cdot \log(0.1) + 1 \cdot \log(0.7) + 0 \cdot \log(0.2)) = \\ &= - (0 + (-0.36) + 0) = \mathbf{0.36} \end{aligned}$$

Плюс и минусы Word2Vec

- Улавливает смысловую близость слов («кошка — собака» ближе, чем «кошка — стол»)
 - Даёт компактные вектора вместо разреженных one-hot
 - Показывает линейные отношения («король — мужчина + женщина $\approx$ королева»)
-
- Один вектор на одно слово, поэтому не различает многозначность («ключ» = источник воды или инструмент)
 - Не учитывает морфологию (работа, работать, работающий — это разные вектора)
 - Контекст ограничен, поэтому слово всегда имеет один и тот же вектор, независимо от предложения

FastText

У алгоритма word2vec есть недостаток: если в новых текстах встречаются слова, отсутствующие в обучающей выборке, их не получится закодировать при помощи word2vec

FastText — это модификация word2vec, которая решает эту проблему при помощи использования символьных N-грамм. То есть в качестве токенов используются не только слова, но и их кусочки, N-граммы



Основная концепция FastText

- Разработан Facebook AI (2016) как улучшение Word2Vec
- Представление слова = сумма векторов его символьных n-грамм
- Вектор слова = эмбеddинг слова + эмбеddинги всех его морфем и частей

Преимущества:

- Можно получить эмбеddинг даже для слова, которого не было в обучающем корпусе

Например: «машинка» разбивается на знакомые куски «машин-» и «-ка»

- Хорошо работает с агглютинативными и морфологически сложными языками:

русский: «работа», «работать», «работающий»

турецкий, финский, венгерский

GloVe

Еще один недостаток word2vec — учет только локального контекста, то есть модель берет в расчет только ближайших соседей слова для его векторизации, при этом игнорируя глобальную структуру корпуса текста. В результате модель может упустить некоторые важные семантические отношения между словами

Модель **GloVe (Global Vectors for Word Representation)** решает эту проблему, объединяя как локальную, так и глобальную статистику появления слов в корпусе текста. Она использует матрицу совместной встречаемости слов, чтобы учесть глобальные закономерности в данных. Такой подход позволяет учитывать не только ближайшие слова, но и все слова в корпусе текста при вычислении векторных представлений

Этапы построения GloVe

1. Первый шаг в GloVe - это создание матрицы совместной встречаемости X . Эта матрица содержит статистику того, как часто слова i и j появляются вместе в корпусе текста. Обычно для этого используется окно контекста - это количество слов слева и справа от данного слова, которое учитывается.
2. Для каждой пары слов i и j значение X_{ij} определяется как частота, с которой слово j появляется в контексте слова i . Матрица совместной встречаемости может быть оценена с использованием эмпирических данных из корпуса текста

Этапы построения GloVe

3. Далее мы вводим некоторую функцию $f(\mathbf{X}_{ij})$, чтобы определить, насколько важна каждая пара слов для модели. Эта функция обычно определяется некоторыми эмпирическими формулами

4. Наконец, при обучении модели мы минимизируем следующую функцию:

$$J = \sum_{i,j=1}^V ((w_i, w_j) - \log(X_{ij}))^2$$

То есть обучаем модель таким векторам, чтобы скалярное произведение слов i и j было близко к логарифму совместной встречаемости слов (суммирование в формуле идет по всем парам слов из словаря размера V)

Пример GloVe

Имеется корпус текста:

Кошка любит молоко

Собака любит молоко

Кошка любит еду

Собака любит еду

Кошка и собака друзья

Кошка пьёт молоко

Собака пьёт молоко

Пример GloVe

Строим матрицу совместной встречаемости (co-occurrence matrix)

слово ↓ / контекст →	кошка	собака	любит	молоко	еду	пьёт	друзья
кошка	–	1	3	2	1	1	1
собака	1	–	3	2	1	1	1
любит	3	3	–	4	2	0	0
молоко	2	2	4	–	0	2	0
еду	1	1	2	0	–	0	0
пьёт	1	1	0	2	0	–	0
друзья	1	1	0	0	0	0	–

Пример GloVe

слово ↓ / контекст →	кошка	собака	любит	молоко	еду	пьёт	друзья
кошка	–	1	3	2	1	1	1
собака	1	–	3	2	1	1	1
любит	3	3	–	4	2	0	0
молоко	2	2	4	–	0	2	0
еду	1	1	2	0	–	0	0
пьёт	1	1	0	2	0	–	0
друзья	1	1	0	0	0	0	–

GloVe “смотрит” на всю эту матрицу и учится превращать каждое слово в вектор (набор чисел, например [0.5, -1.2, 0.8, ...]), так чтобы расстояния между векторами отражали смысловую близость.

Например, вектора слов "молоко" и "любит" будут ближе друг к другу, чем "друзья" и "еда", потому что они больше похожи по смыслу

GloVe и Word2Vec

Оба метода используют окно контекста для определения связей

Но Word2Vec работает локально, обучаясь на каждом предложении, а GloVe сначала собирает всю статистику, а потом делает факторизацию матрицы

- Word2Vec: смотрит на текущее предложение и меняет вектора
- GloVe: собирает общую картину встречаемости слов в языке, а потом обучает вектора

Свойства эмбедингов

- Семантическая близость — похожие слова имеют близкие вектора (кошка — собака)
- Кластеризация — слова группируются по тематикам (животные, города, глаголы)
- Арифметика смыслов — линейные отношения (король — мужчина + женщина $\approx$ королева).
- Синонимия и морфология — похожие формы слов тяготеют друг к другу (машина — автомобиль)
- Контекстность (в современных моделях) — разные значения слова отражаются в разных векторах (ключ к задаче $\neq$ ключ от замка)

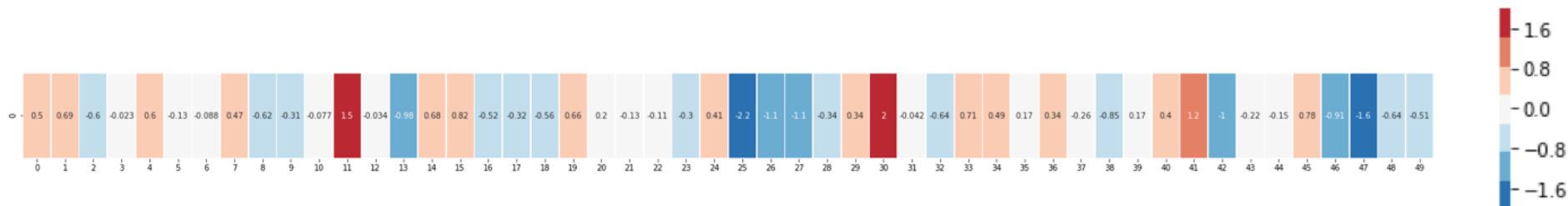
Свойства эмбедингов

Эмбединг слова “король”, полученный с помощью GloVe на текстах из Википедии:

[0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 ,
0.47377 , -0.61798 , -0.31012 , -0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 ,
0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 ,
-0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 , -
0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 ,
0.1661 , 0.40102 , 1.1685 , -1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -
1.6106 , -0.64426 , -0.51042]

Свойства эмбедингов

Визуализируем с помощью heatmap (тепловой карты)



Свойства эмбедингов

Визуализируем с помощью heatmap (тепловой карты)

“king”



“Man”



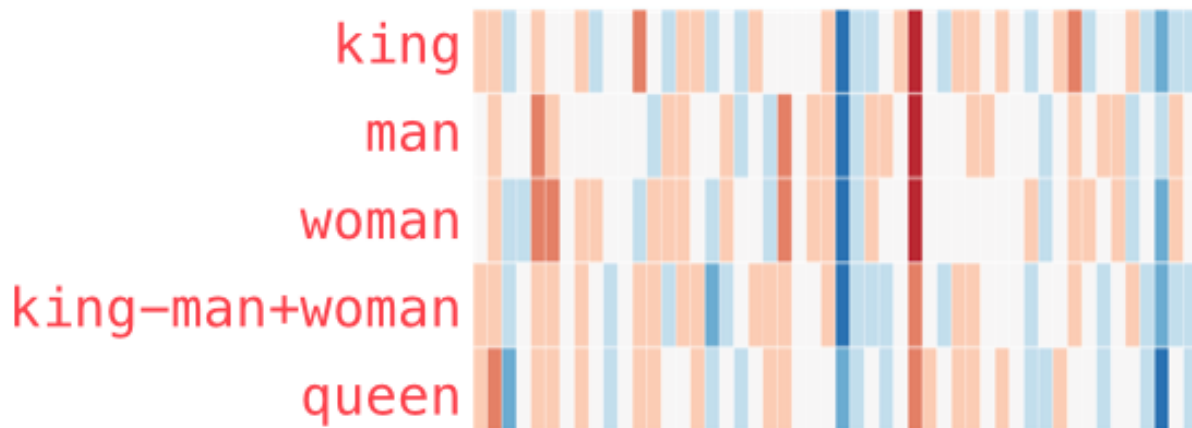
“Woman”



Свойства эмбедингов

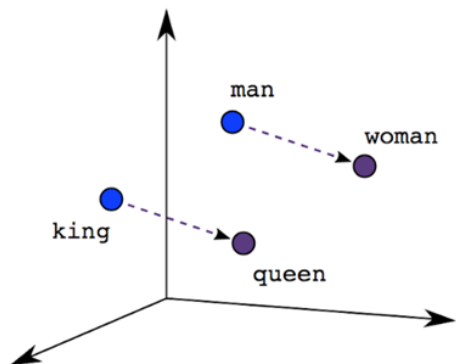
Вычтем из слова “король” слово “мужчина” и прибавим слово “женщина”

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

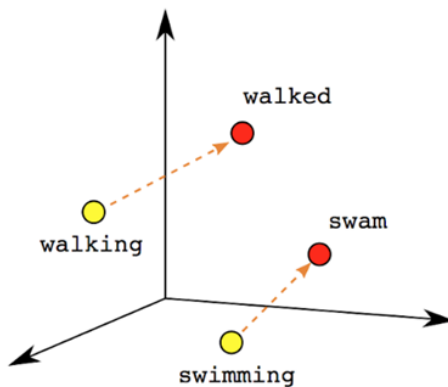


Наиболее близкий результат в словаре из 400 000 слов

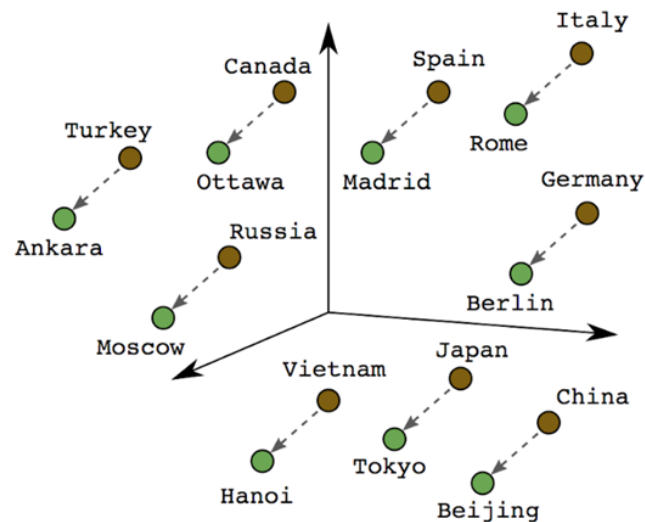
Визуализация в многомерном пространстве



Male-Female



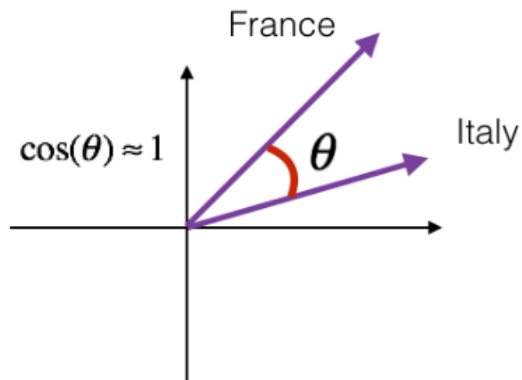
Verb Tense



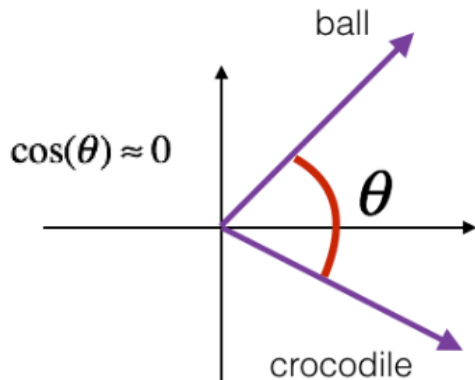
Country-Capital

Семантическое сходство эмбедингов

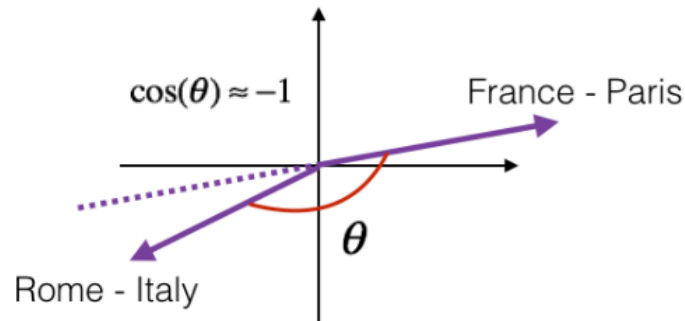
Косинусное сходство: $\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$



Франция и Италия схожи по смыслу



Мяч и крокодил не похожи по смыслу



Вектор “Франция минус Париж” и вектор “Рим минус Италия” похожи по смыслу, но имеют разный знак